

PROGRAMMIERUNG 2

Kapitel 0: **Organisatorisches**



Prof. Dr. Markus Esch
htw saar

Dozent



- ▶ Markus Esch
 - ▶ Professor für Softwarearchitekturen und Verteilte Systeme
 - ▶ Studienleiter Praktische Informatik
 - ▶ Leiter Distributed Systems Lab

▶ Werdegang

- ▶ Studium Diplom-Informatik Universität Trier
- ▶ Doktorand Universität Luxemburg
- ▶ Postdoc Universität Luxemburg und Université catholique de Louvain
- ▶ Forschungsgruppenleiter Fraunhofer FKIE
- ▶ Lehrbeauftragter Universität Bonn
- ▶ seit WS 2016: Professor htw saar



 **Universität Trier**


UNIVERSITÉ DU
LUXEMBOURG



UCL

 **Fraunhofer**

FKIE

htw saar


universität **bonn**



Allgemeine **Informationen**

- ▶ Vorlesungen: Prof. Dr. Markus Esch
- ▶ Übungen:
 - ▶ Emma Büch
 - ▶ Michael Meßner
 - ▶ Moritz Niederer
 - ▶ Regina Piontek
 - ▶ Andreas Schaffhauser
- ▶ Hörerkreis: PI Bachelor, DFHI Informatik Bachelor
- ▶ Lehrform: Vorlesungen (4 SWS) + Übung (2 SWS)
- ▶ 8 ECTS

Zeiten

► **Vorlesungen (Hörsaal 5206)**

- Dienstag 11:45 - 13:15
- Dienstag 14:15 - 15:45

► **Übungen (Raum 8207)**

- Dienstag 10:00 - 11:30 (DFHI)
- Mittwoch 08:15 - 09:45
XOR
- Mittwoch 10:00 - 11:30
XOR
- Mittwoch 11:45 - 13:15



Lernziele der Vorlesung



Themenbereich Java

- ▶ Die Studierenden können:
 - ▶ fortgeschrittene Konzepte objektorientierter und funktionaler Programmierung in Java erklären und anwenden
 - ▶ objektorientierte Lösungen entwerfen und umzusetzen

Lernziele der Vorlesung



Themenbereich Algorithmen und Datenstrukturen

- ▶ Die Studierenden können:
 - ▶ verschiedene Datenstrukturen implementieren und anwenden
 - ▶ die für ein Anwendungsszenario geeignete Datenstruktur auszuwählen und anwenden

Lernziele der Vorlesung



Themenbereich C

- ▶ Die Studierenden können:
 - ▶ Konzepte und Eigenheiten der imperativen Programmierung in C benennen
 - ▶ Unterschiede sowie Vor- und Nachteile von C zu objektorientierten und typsicheren Sprachen erläutern
 - ▶ einfache C-Programme implementieren

Ausblick: **Inhalte**

► ***Themenbereich Java***

- Rekursion
- Innere Klassen
- Lambda-Ausdrücke
- Generics
- (Annotations)
- (Reflection)
- (Optionals)



Ausblick: **Inhalte**

► **Themenbereich Algorithmen und Datenstrukturen**

- Implementierungsaspekte von Bäumen, Graphen und Listen
- Java Collection Framework



Ausblick: **Inhalte**

► **Themenbereich C**

- Struktur eines C-Programms
- Ausdrücke, Operatoren, Kontrollstrukturen und Funktionen
- Einfache- und strukturierte Datentypen
- Pointer und Pointer-Arithmetik
- Speicherverwaltung
- Präprozessor, Compiler, Linker, Debugger und make
- Nutzung von Bibliotheken
- Komplexe Datenstrukturen in C



Zeitplanung Vorlesung

Termin	Thema
07.04.26	Organisatorisches / OO in a Nutshell
14.04.26	Rekursion
21.04.26	Campustage (keine Vorlesung)
28.04.26	Innere Klassen
05.05.26	Generics
12.05.26	Generics / Lambda-Ausdrücke
19.05.26	Lambda-Ausdrücke
26.05.26	Lambda-Ausdrücke / Collections
02.06.26	Collections
09.06.26	Collections
10.06.26 (Zusatztermin, 1. Stunde)	Collections
16.06.26	C-Einführung / C-Grundlagen
23.06.26	C-Programmstruktur
30.06.26	C-Aggregattypen
01.07.26 (Zusatztermin, 4. Stunde)	C-Pointer
07.07.26	C-Pointer
14.07.26	keine Vorlesung

Zeitplanung Übungen

Termin	Thema
08.04.26	
15.04.26	Abgabe Ü1: Wiederholung & Einführung IntelliJ IDEA
22.04.26	
29.04.26	Abgabe Ü2: Rekursion
06.05.26	Abgabe Ü3: Innere Klassen
13.05.26	Abgabe Ü4: Generics
20.05.26	
27.05.26	Abgabe Ü5: Lambda-Ausdrücke 1
03.06.26	Abgabe Ü6: Lambda-Ausdrücke 2
10.06.26	
17.06.26	Abgabe Ü7: Collections 1
24.06.26	Abgabe Ü8: Collections 2
01.07.26	Abgabe Ü9: C-Grundlagen
08.07.26	Abgabe Ü10: C-Programmstruktur
15.07.26	Abgabe Ü11: Aggregattypen und Pointer

Arbeitsaufwand

- ▶ 8 ECTS —> **240** Stunden!!!
- ▶ Präsenzzeit:
 - ▶ 4 SWS Vorlesung + SWS Übung
 - ▶ 90 Veranstaltungsstunden
= 67,5 Zeitstunden
- ▶ Vor-/Nachbereitung, Prüfungsvorbereitung
 - ▶ 172,5 Stunden
 - ▶ **11,5** Stunden pro Woche!



Übungen **Allgemeines**

- ▶ Es gibt **11** Übungsblätter.
 - ▶ Für jedes Übungsblatt gibt es einen Abgabetermin.
 - ▶ Zu jeder Übung wird ein **Lernziel** formuliert.
 - ▶ Zu min. **9** Übungsblätter benötigen Sie ein **Review**
- ▶ An Übungstagen ohne Abgabe findet Beratung statt.



Coding **Teams**

- ▶ Bilden Sie selbständig 2er-Teams und melden Sie sich in Moodle zu einer Übungsgruppe an.



- ▶ Frist: **12.04.26**
 - ▶ Eine verspätete Anmeldung ist nicht möglich!



Code **Reviews**

- ▶ Jedes Coding-Team muss zu min. **9** von **11** Übungsblättern ein **Review** erhalten haben. **Mindestens vier** der Reviews müssen **Tutoren-Reviews** sein, die restlichen **Peer-Reviews**.
 - ▶ Jedes Review wird durch das Team, welches das Review erhalten hat, in Form eines **Reports** dokumentiert.
 - ▶ Es ist **ihre** Verantwortung die ausreichende Anzahl an Tutoren-Reviews zu erhalten!
 - ▶ Es wird **empfohlen**, Peer-Reviews in den Übungsstunden durchzuführen.
 - ▶ bei Fragen können Tutoren helfen

➔ Details siehe **Leitfaden Code Reviews** in Moodle



Ablauf Reviews (1/2)

- ▶ Präsentieren Sie einen Code Walkthrough von **5 bis max. 10 Minuten**. Gehen Sie dabei auf folgende Aspekte ein:
 - ▶ Wie funktioniert ihre Lösung?
 - ▶ Inwiefern setzt die Lösung die Anforderungen der Aufgabenstellung um?
 - ▶ Warum wurde die Aufgabe so gelöst? Hätte es alternative Implementierungsmöglichkeiten gegeben?
 - ▶ Gibt es Teile der Aufgabenstellung, die nicht umgesetzt werden konnten?
 - ▶ Was hat bei der Implementierung gut funktioniert und wo gab es Schwierigkeiten?
 - ▶ Welche Fragen und Probleme sind noch offen?



Ablauf Reviews (2/2)

- ▶ Anschließend:
 - ▶ Klärung von Fragen der Studierenden
 - ▶ Diskussion der Vor- und Nachteile der Lösung
 - ▶ Verbesserungsvorschläge
 - ▶ Erörterung von Detailfragen
- ▶ Ist ein Code Walkthrough unzureichend vorbereitet oder wurde die Lösung offensichtlich nicht selbst implementiert entfällt die anschließende Diskussion.
 - ➡ Mehr Zeit für engagierte Studierende.
- ▶ Kriterium für das Bestehen eines Reviews ist die Durchführung des Code Walkthroughs und das Einreichen des Reports.



Review-Report zur Übung Nr. ...

Coding-Team:

Review erhalten von:

Welche spezifischen Aspekte Ihres Codes wurden im Review positiv hervorgehoben? Warum wurden sie Ihrer Meinung nach hervorgehoben? Welche guten Praktiken spiegeln diese Aspekte wider?

- ...

Erläutern Sie, welche Anforderungen als unvollständig oder nicht umgesetzt identifiziert wurden und welche Hinweise ihnen gegeben wurden, um die Anforderungen umzusetzen:

- ...

Welche Hinweise wurden Ihnen gegeben, um den Code sauberer, lesbarer und wartbarer zu gestalten?

- ...

Welche weiteren konkreten Ratschläge haben Sie erhalten und wie können diese die Qualität Ihrer Implementierung verbessern?

- ...

Welche Fragen zu Ihrem Code konnten Sie während des Reviews nicht beantworten und wie werden Sie diese Unklarheiten klären?

- ...

Welche Aspekte der Übung sind Ihnen unklar geblieben und welche Schritte werden Sie unternehmen, um diese zu verstehen?

- ...

Bewerten Sie, inwieweit Sie die Lernziele der Übung erreicht haben. Welche Lernziele

Review Report

- ▶ Template Review Report in Moodle
- ▶ fristgerechte Abgabe des Reports in Moodle
(Donnerstag 19:00 Uhr)
 - ▶ verspätete Abgaben sind nicht möglich!
- ▶ Stichprobenprüfung der Reports
 - ▶ zwei Mal je Team im Semester
 - ▶ **mangelhafte Reviews werden nicht akzeptiert!**

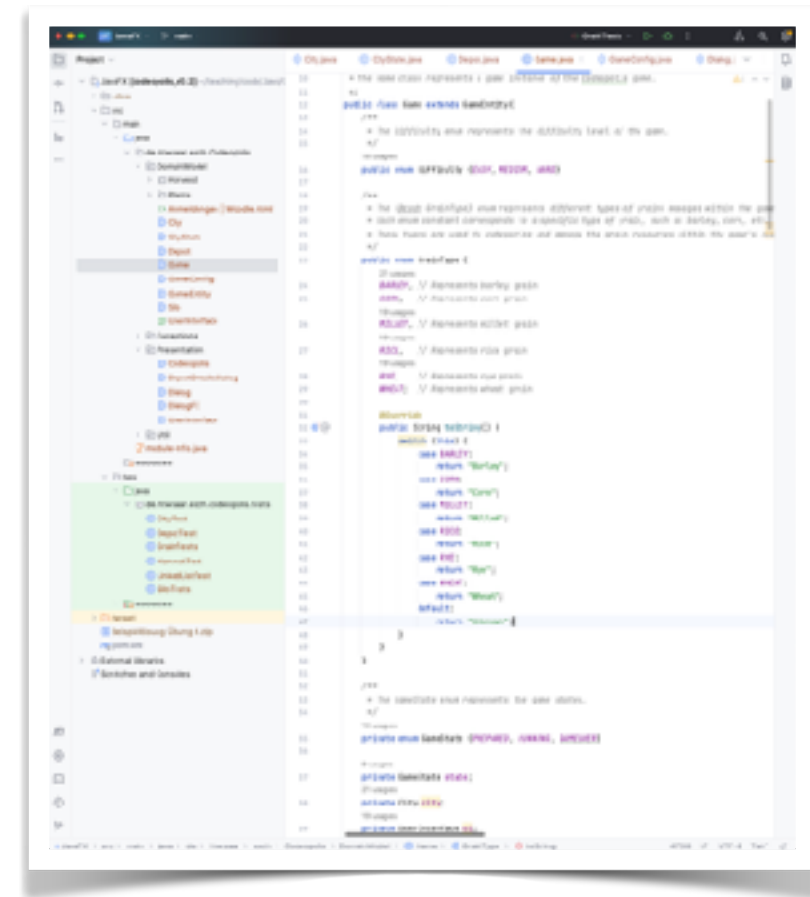
Ziel der **Reviews**

- ▶ **Erkenntnisgewinn** nicht Überprüfung
- ▶ Sie sollen ...
 - ▶ ihre eigene Lösung reflektieren
 - ▶ wertvolle Tipps und Hinweise erhalten
 - ▶ Hilfestellung bei Problemen erhalten
 - ▶ sich verbessern
 - ▶ etwas lernen
- ▶ Voraussetzung: Sie bearbeiten die Übungen gewissenhaft und selbstständig.
 - ➔ **Wer generierte Lösungen verwendet betrügt sich selbst!**



Einführung **IntelliJ IDEA**

- Übung in der zweiten Semesterwoche



Prüfungsleistung

- ▶ Prüfungsart: Klausur
 - ▶ Termin wird noch bekannt gegeben
- ▶ Voraussetzungen Klausurzulassung:
 - ▶ Reviews zu **9** von **11** Übungsblättern
 - ▶ davon **4** Tutorenreviews.
 - ▶ fristgerechte Abgabe des Review-Reports in Moodle
- ▶ Klausurzulassungen aus früheren Semestern bleiben gültig



Krankheitsfälle

- ▶ Mit der Vorgabe “nur” 9 von 11 Übungen abgeben zu müssen, sind Krankheitsfälle in der Regel abgedeckt, Sie müssen dann keinen Attest einreichen.
- ▶ Wenn Sie mehr als 2 Übungen krankheitsbedingt verpassen, müssen Sie einen Attest für **ALLE** verpassten Übungen einreichen.



Ausnahmeregelungen

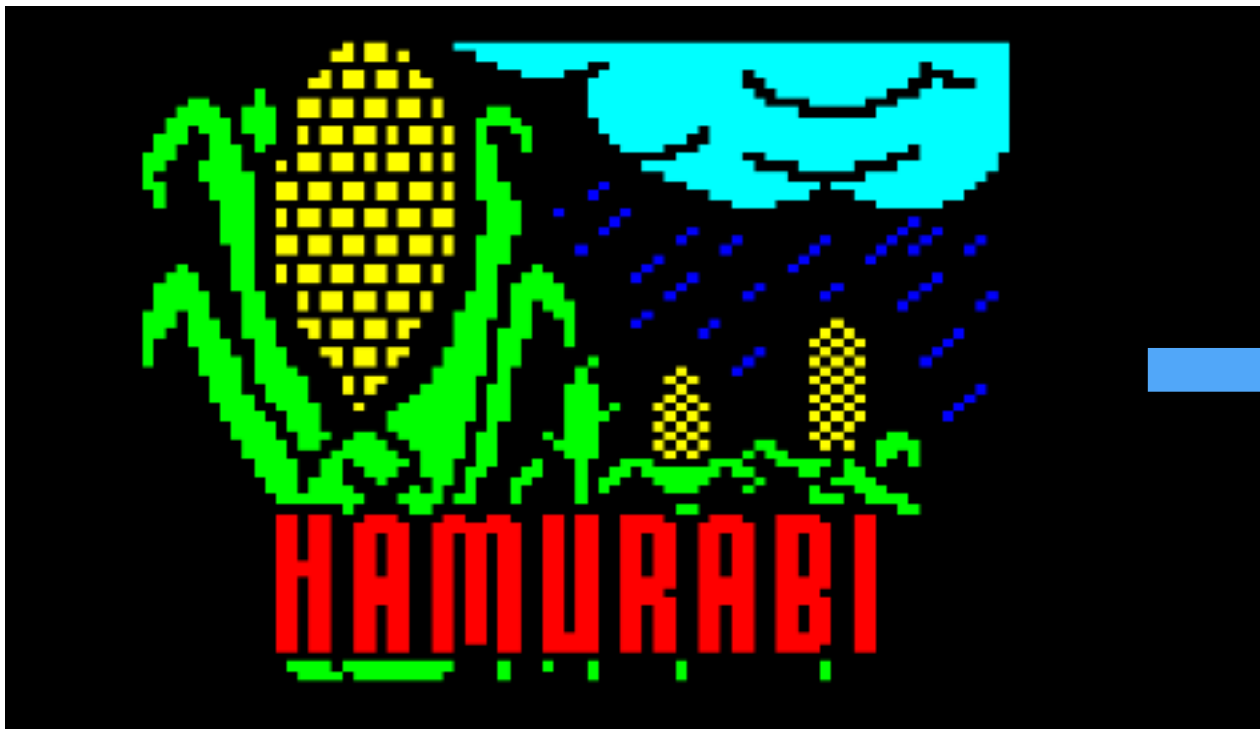
- ▶ Alle Fristen und Regeln sind verbindlich
- ▶ Ausnahmen von den Regeln nur in **sehr gut begründeten** Einzelfällen!!!
 - ▶ Beantragung von Ausnahmen **ausschließlich** per Email an den Modulverantwortlichen
 - ▶ bei Anfragen mit Bezug zu Übungen, alle Tutoren in CC



CODEOPOLIS



Codeopolis



- ▶ textbasierte Wirtschaftssimulation
- ▶ 1968
- ▶ FOCAL



- ▶ Java-Portierung
- ▶ Kofferwort: Code + Metropole
- ▶ Programmierkonzepte anwenden
- ▶ Zusammenhänge verstehen

Programmierung 2 - Sommersemester 2025

Prof. Dr. Markus Esch

Übung Nr. 1
Abgabe KW 16

Hamurabi ist eine textbasierte Wirtschaftssimulation aus dem Jahr 1968, die ursprünglich in FOCAL geschrieben und später nach BASIC portiert wurde. Wikipedia beschreibt den Spielablauf wie folgt: In Hamurabi verwaltet der Spieler als König einen antiken Stadtstaat. Ziel des Spiels ist es, innerhalb von zehn Runden möglichst wenige Hungertote zu verursachen und das Gebiet des Staates auszuweiten. Dabei ist jede Runde neu zu entscheiden, wie viele Morgen Land gekauft oder verkauft werden und wie viele Scheffel Getreide an die Bevölkerung als Nahrung ausgegeben oder als Saat für die kommende Ernte verwendet werden sollen. Spielbeeinflussende Zufallsfaktoren sind ein variierender Ernteertrag pro Morgen und ein sich verändernder Landpreis. Als weitere Zufallseignisse können eine Landplage auftreten, die die Bevölkerung dezimiert, und Ratten, die eine bestimmte Menge des gelagerten Getreides auffressen. Sollte der Spieler nicht vor Ablauf der zehnten Runde von seiner Bevölkerung abgesetzt oder ermordet worden sein, erhält er am Ende eine leicht humoristisch anmutende Beurteilung seiner Leistung (Quelle: <https://de.wikipedia.org/wiki/Hamurabi>).

Heute gibt es verschiedene Portierungen des Spiels, so dass es auch im Browser gespielt werden kann, unter anderem hier:

<https://adventuron.it.ch.io/hamurabi>

Probieren Sie das Spiel einmal aus, das Verständnis des Spielablaufs wird Ihnen die Umsetzung sicherlich erleichtern.

Im *Codeopolis*-Projekt wollen wir eine Java-Version des Spiels entwickeln. Wir wollen uns aber nicht mit dem ursprünglichen Funktionsumfang von Hamurabi begnügen, sondern werden das Spiel im Laufe der Zeit deutlich erweitern. *Codeopolis* ist ein Kofferwort, das die Begriffe *Code* und *Metropolis* kombiniert. Es verbindet die Welt der Programmierung mit dem Spielziel, eine Stadt zu einer blühenden Metropole zu entwickeln.

Am Ende des ersten Semesters haben Sie für die vorlesungsfreie Zeit eine Reihe von freiwilligen Übungen zum Codeopolis-Projekt erhalten. Einige von Ihnen haben diese Aufgaben vielleicht bearbeitet. In den Übungen wurden alle Themen von Programmierung 1 wiederholt. In Programmierung 2 werden wir an dieser Stelle fortfahren und die im Laufe des Semesters neu erlernten Programmier Techniken zur Erweiterung des Spiels nutzen. Dies soll Ihnen die Möglichkeit geben, die im Studium erlernten Programmier Techniken im Kontext eines größeren Programmierprojektes anzuwenden und so den Gesamtzusammenhang zu erkennen.

Übung 1

Abgabe KW 17

- Aufgabe 1: Codeopolis
Basislösung für
Programmierung 2
verstehen
- Aufgabe 2: Kleine
Refactoring-Aufgabe

Programmierung 2 - Sommersemester 2025

Prof. Dr. Markus Esch

Übung Nr. 1
Abgabe KW 16

Hamurabi ist eine textbasierte Wirtschaftssimulation aus dem Jahr 1968, die ursprünglich in FOCAL geschrieben und später nach BASIC portiert wurde. Wikipedia beschreibt den Spielablauf wie folgt: In Hamurabi verwaltet der Spieler als König einen antiken Stadtstaat. Ziel des Spiels ist es, innerhalb von zehn Runden möglichst wenige Hungertote zu verursachen und das Gebiet des Staates auszuweiten. Dabei ist jede Runde neu zu entscheiden, wie viele Morgen Land gekauft oder verkauft werden und wie viele Scheffel Getreide an die Bevölkerung als Nahrung ausgegeben oder als Saat für die kommende Ernte verwendet werden sollen. Spielbeeinflussende Zufallsfaktoren sind ein variierender Ernteertrag pro Morgen und ein sich verändernder Landpreis. Als weitere Zufallseignisse können eine Landplage auftreten, die die Bevölkerung dezimiert, und Ratten, die eine bestimmte Menge des gelagerten Getreides auffressen. Sollte der Spieler nicht vor Ablauf der zehnten Runde von seiner Bevölkerung abgesetzt oder ermordet worden sein, erhält er am Ende eine leicht humoristisch anmutende Beurteilung seiner Leistung (Quelle: <https://de.wikipedia.org/wiki/Hamurabi>).

Heute gibt es verschiedene Portierungen des Spiels, so dass es auch im Browser gespielt werden kann, unter anderem hier:

<https://adventuron.itch.io/hamurabi>

Probieren Sie das Spiel einmal aus, das Verständnis des Spielablaufs wird Ihnen die Umsetzung sicherlich erleichtern.

Im *Codeopolis*-Projekt wollen wir eine Java-Version des Spiels entwickeln. Wir wollen uns aber nicht mit dem ursprünglichen Funktionsumfang von Hamurabi begnügen, sondern werden das Spiel im Laufe der Zeit deutlich erweitern. *Codeopolis* ist ein Kofferwort, das die Begriffe *Code* und *Metropolis* kombiniert. Es verbindet die Welt der Programmierung mit dem Spielziel, eine Stadt zu einer blühenden Metropole zu entwickeln.

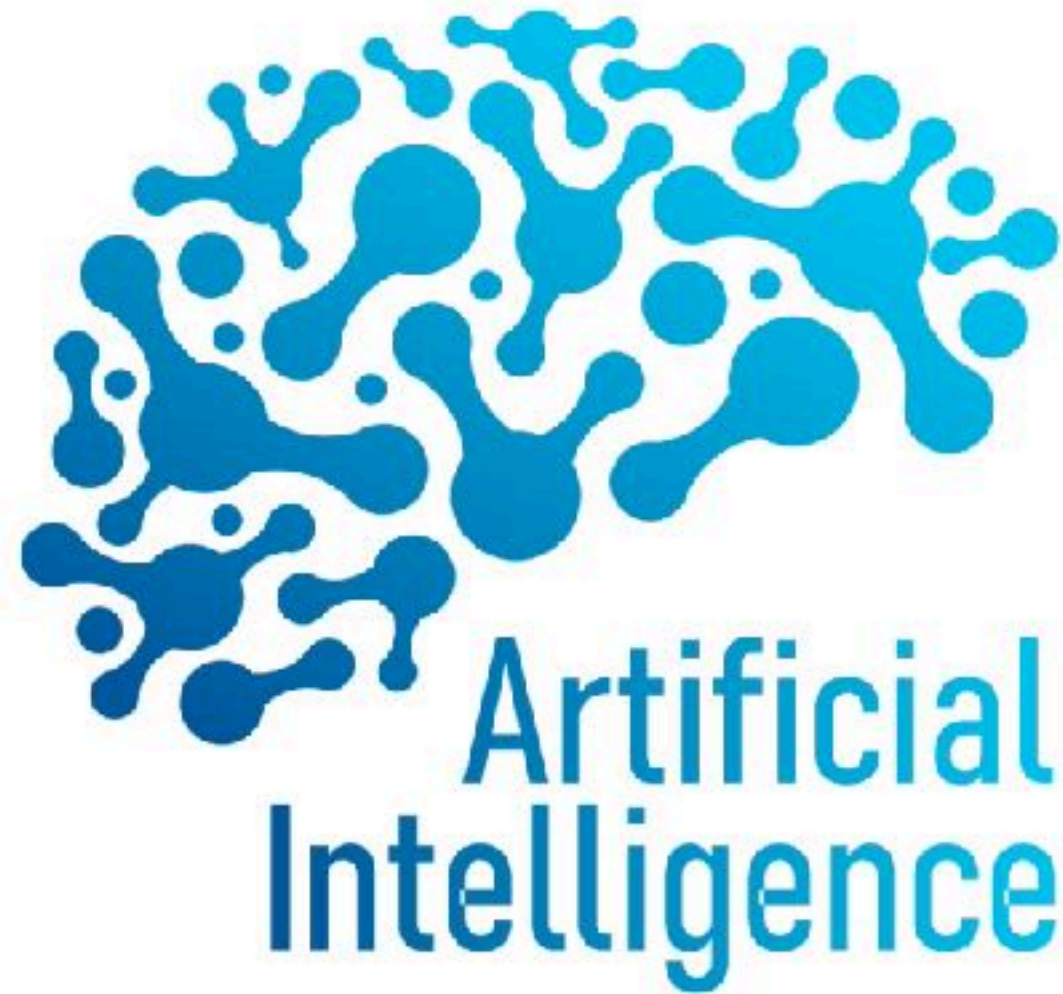
Am Ende des ersten Semesters haben Sie für die vorlesungsfreie Zeit eine Reihe von freiwilligen Übungen zum Codeopolis-Projekt erhalten. Einige von Ihnen haben diese Aufgaben vielleicht bearbeitet. In den Übungen wurden alle Themen von Programmierung 1 wiederholt. In Programmierung 2 werden wir an dieser Stelle fortfahren und die im Laufe des Semesters neu erlernten Programmier Techniken zur Erweiterung des Spiels nutzen. Dies soll Ihnen die Möglichkeit geben, die im Studium erlernten Programmier Techniken im Kontext eines größeren Programmierprojektes anzuwenden und so den Gesamtzusammenhang zu erkennen.

Übung 1

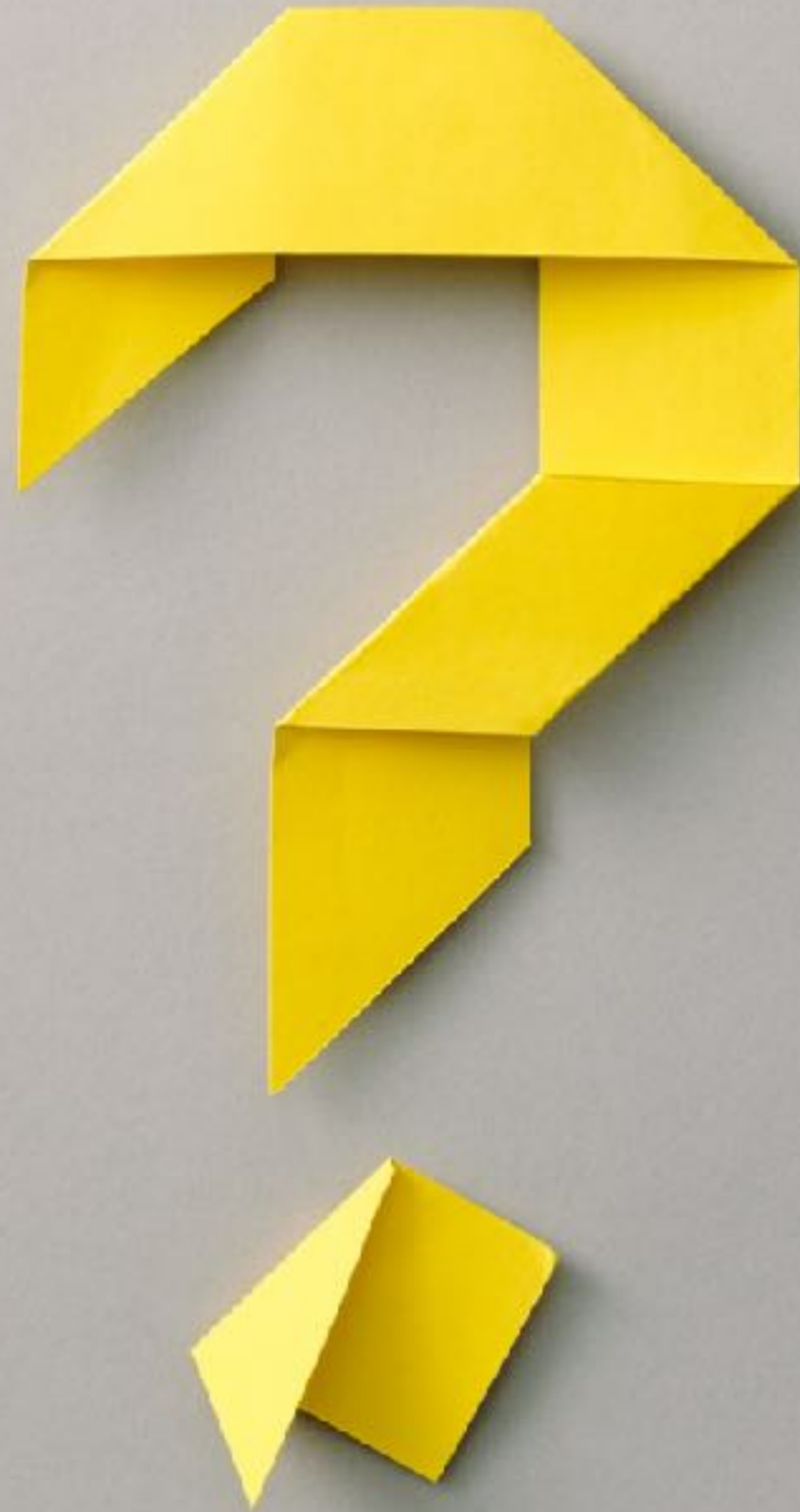
Lernziele

- ▶ Nach der Bearbeitung des Übungsblattes sollten Sie ...
 - ▶ ... die Inhalte aus Programmierung 1 erfolgreich rekapituliert haben.
 - ▶ ... in der Lage sein, die Struktur der Baseline-Lösung zu erläutern.
 - ▶ ... erklären können, was die einzelnen Klassen und Methoden der Baseline-Lösung machen und wie sie zusammenhängen.
 - ▶ ... den Kontrollfluss erklären können.
 - ▶ ... erkannt haben, wie die in Programmierung 1 erlernten Programmier Techniken angewendet wurden.
 - ▶ ... in der Lage sein, eine kleine Anpassung des Codes vorzunehmen.
 - ▶ ... den Code so gut verstanden haben, dass sie ihn in den nächsten Übungen weiterentwickeln können.

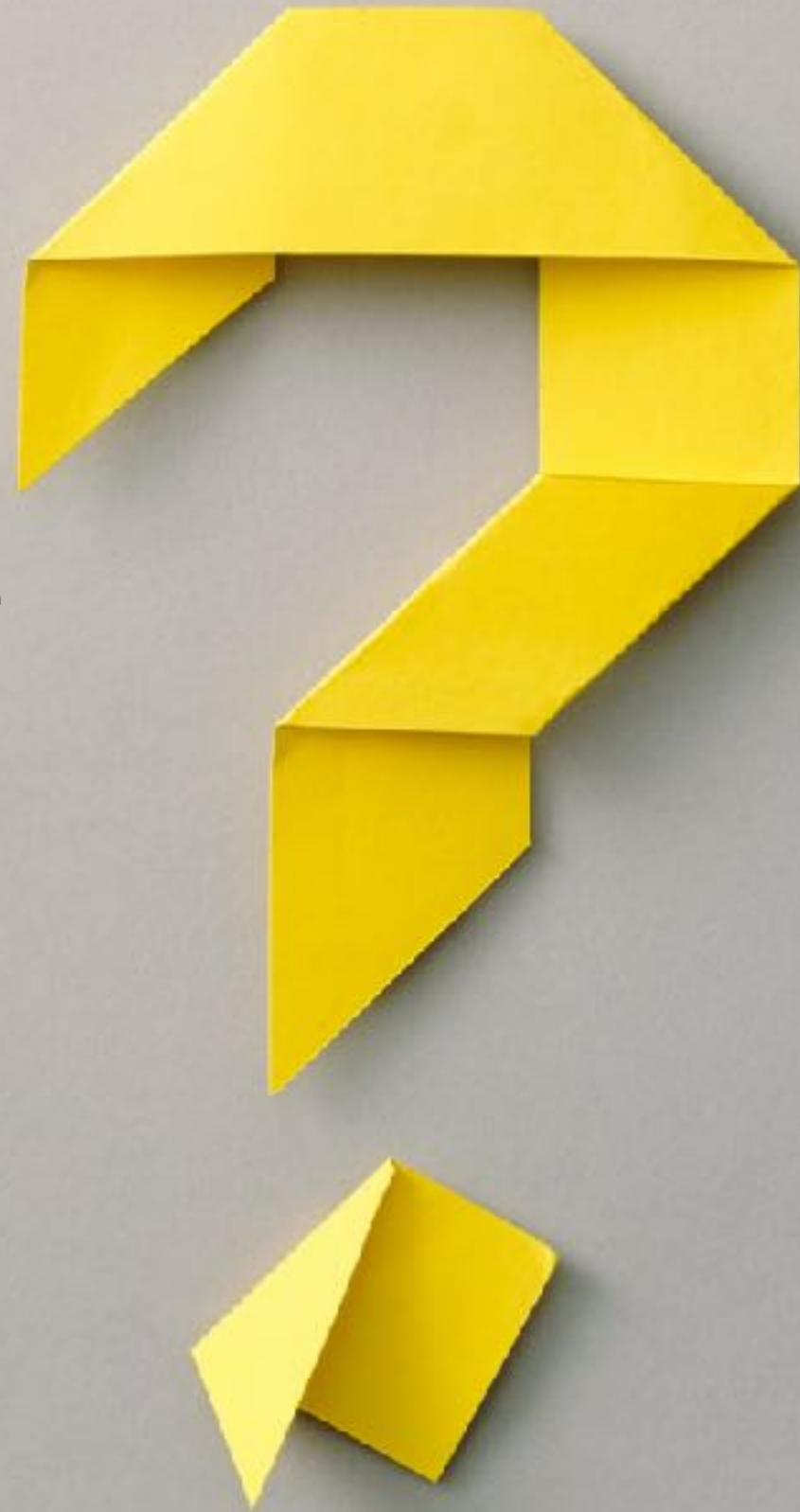
KI-Assistenten beim Programmieren lernen



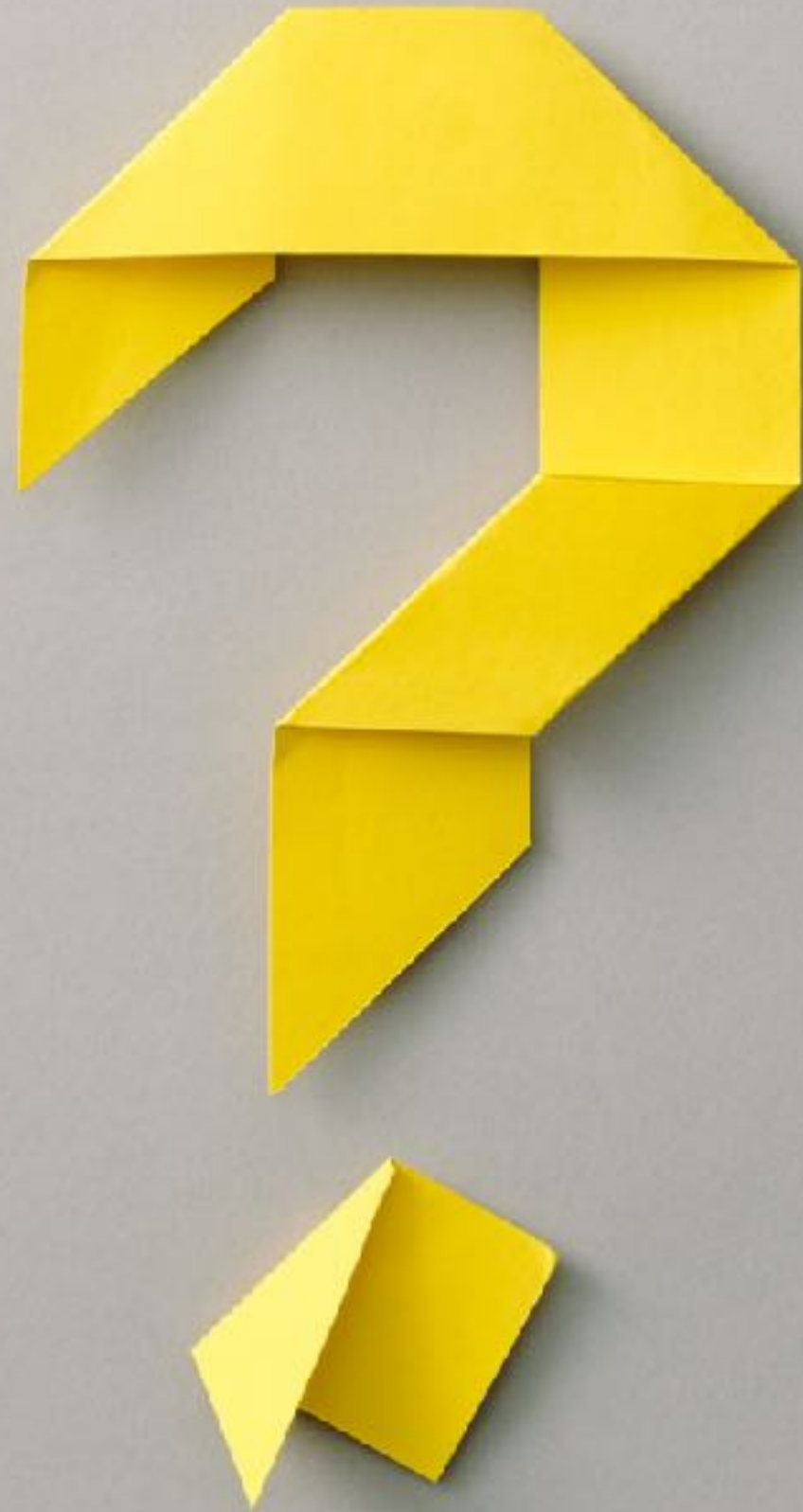
**Sind Taschenrechner ein
nützliches Werkzeug?**



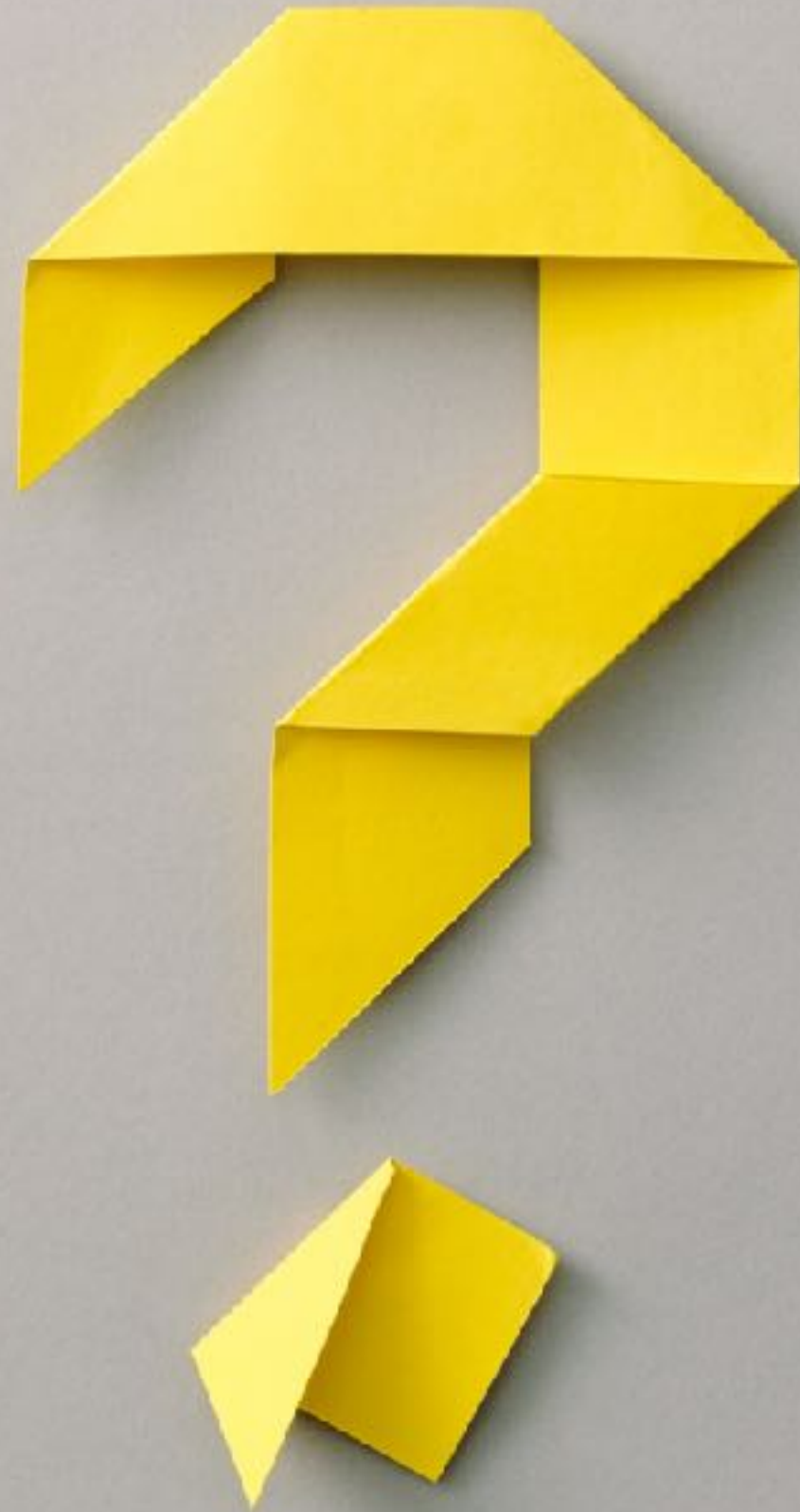
**Sollten Kinder in der Grundschule
ihre Matheaufgaben mit einem
Taschenrechner lösen?**



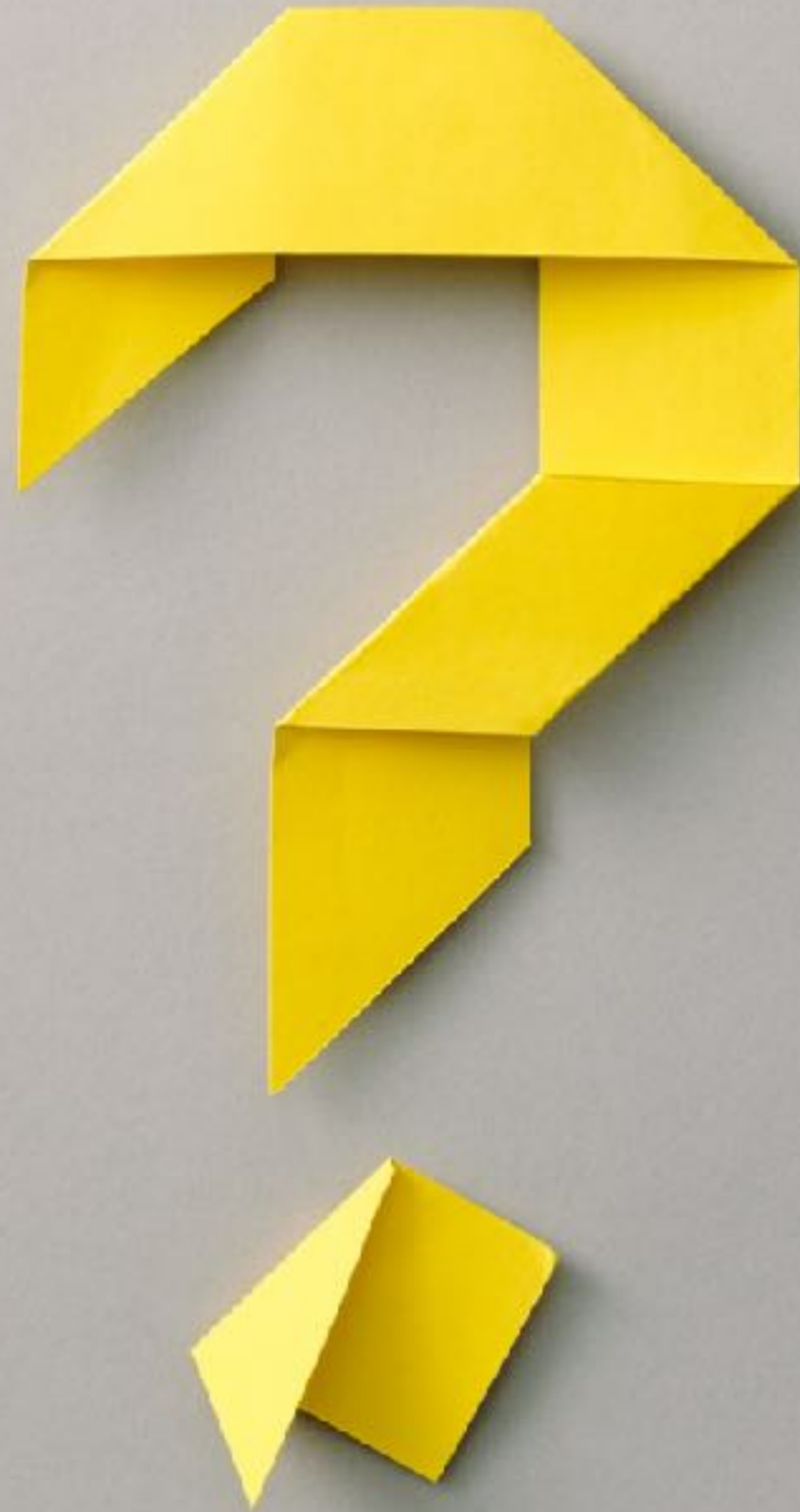
**Sind KI-Assistenten ein
nützliches Werkzeug, um
Software-Entwickler zu
unterstützen?**



**Sollte Studierende ihre
Programmieraufgaben in den
ersten Semestern mit KI-
Assistenten lösen?**



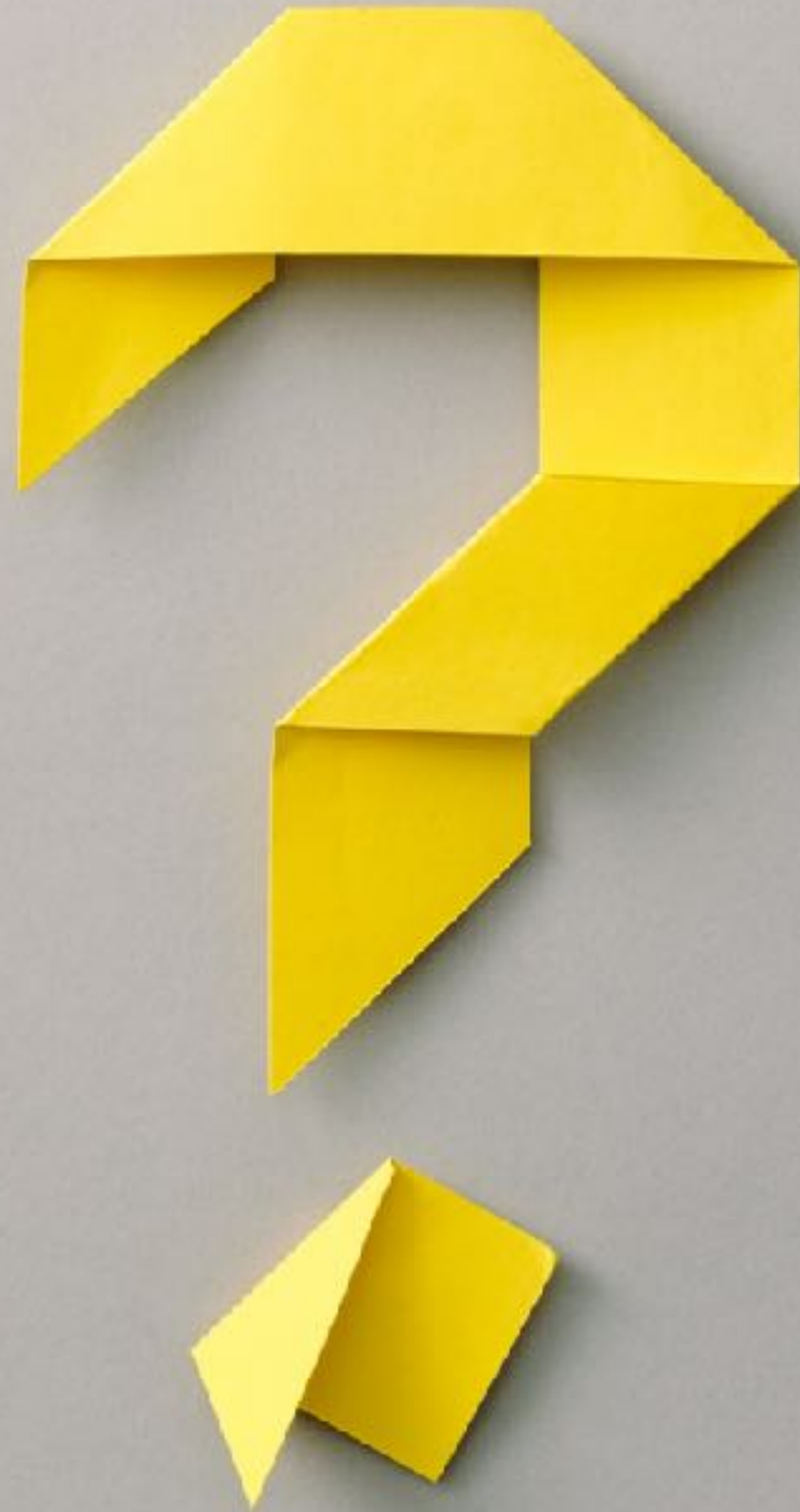
**Sollten Menschen die
Entwicklung von Software
zukünftig der KI überlassen?**



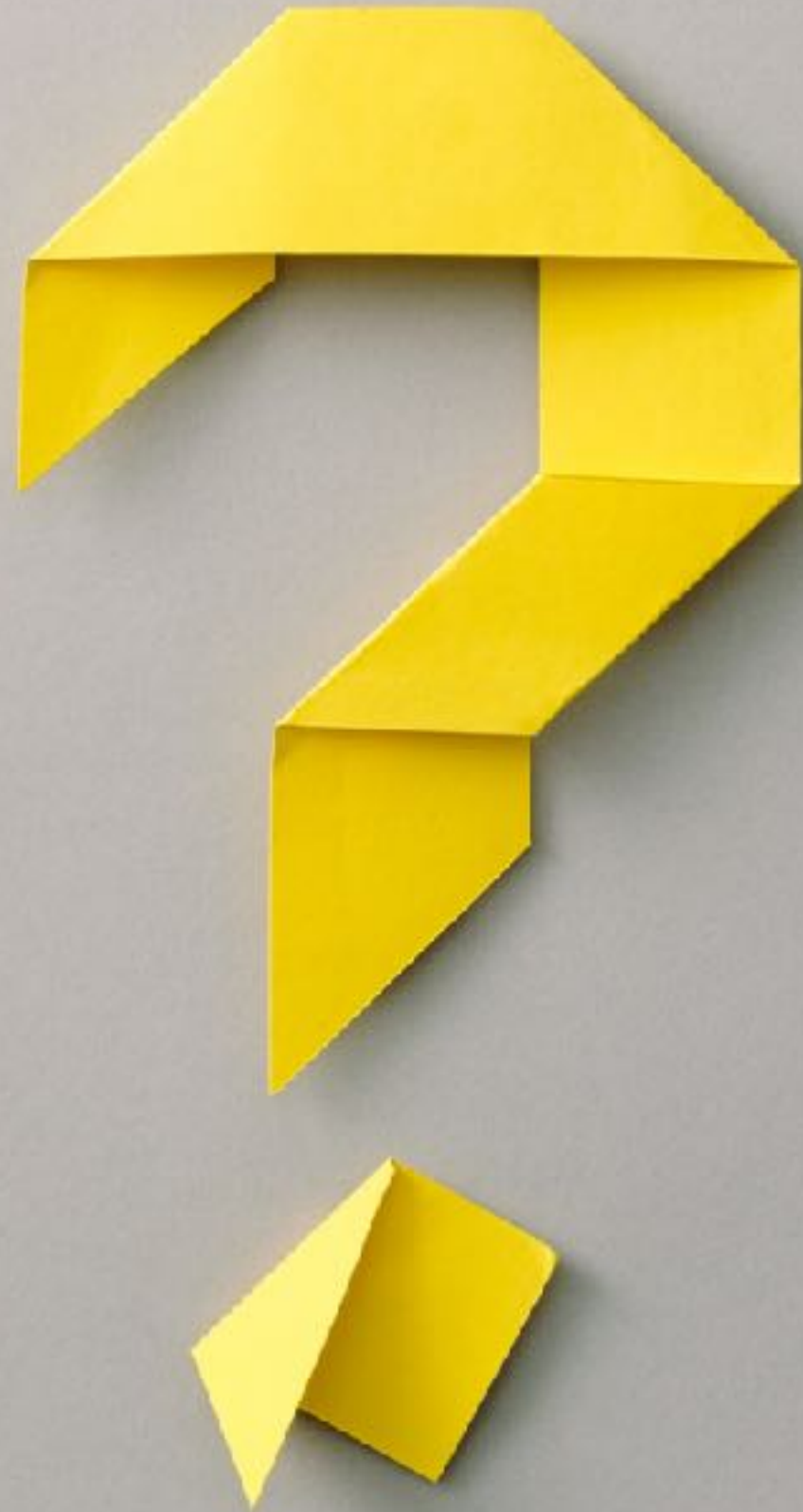
Kann man KI-Assistenten ohne fundierte Programmierkenntnisse und -erfahrung zielgerichtet, effizient und SICHER einsetzen?



**Erlangt man fundierte
Programmierkenntnisse und
-erfahrung wenn man selbst
einfache Übungen nicht mehr
selbst macht?**



Werden Software-Entwickler auf dem Niveau eines KI-Assistenten zukünftig noch benötigt?



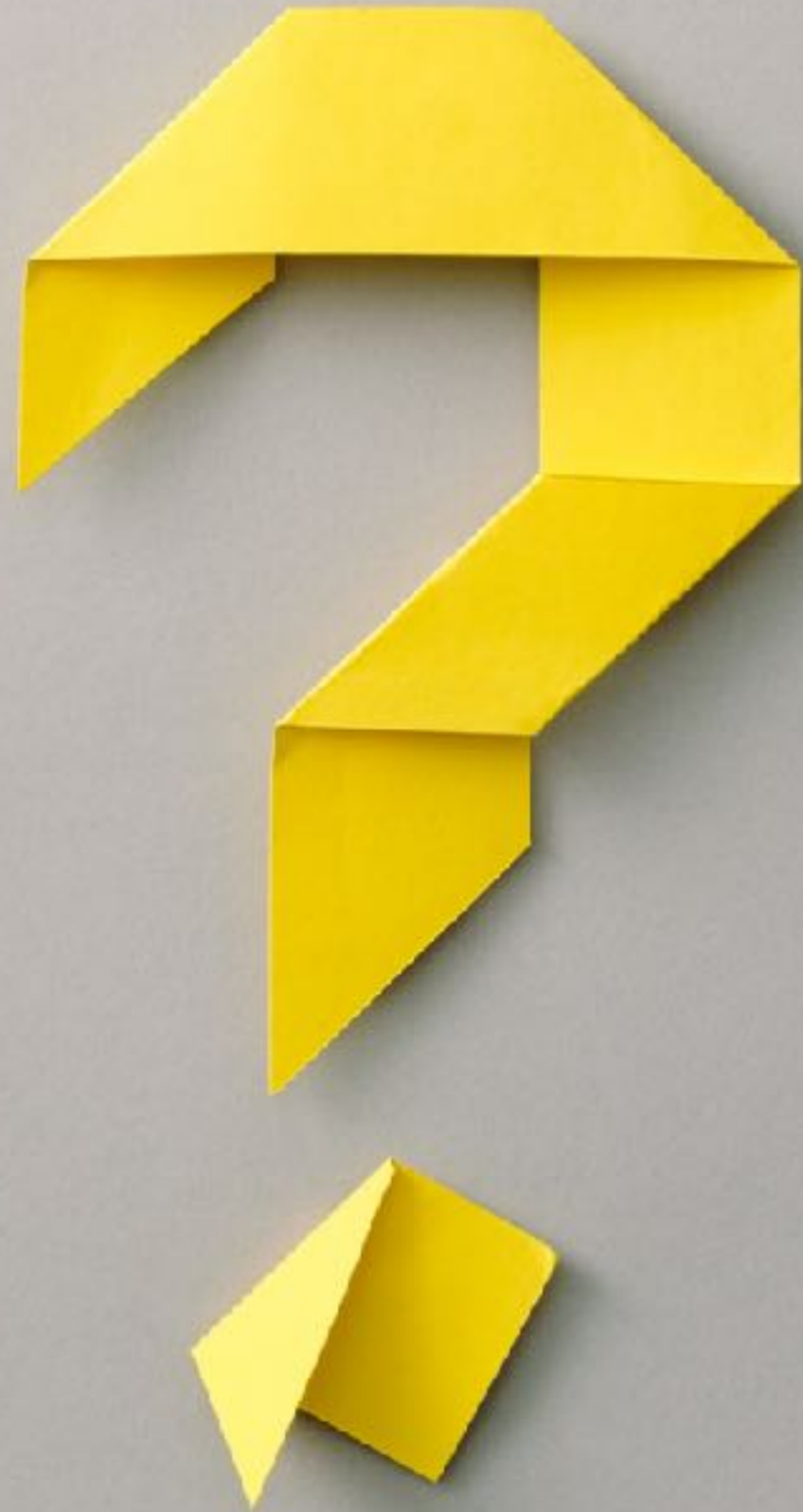
Chat GPT, auf welchem Niveau kannst du programmieren?



Wenn man menschliche Erfahrungsstufen als Orientierung nimmt (z. B. Junior-, Mid- und Senior-Entwickler), bin ich am ehesten mit einer Mischung aus Junior- und Mid-Level-Kenntnissen vergleichbar.



**Werden Sie ohne eigenes Üben,
Ausprobieren sowie Fehler
machen und korrigieren besser
als ein KI-Assistent?**

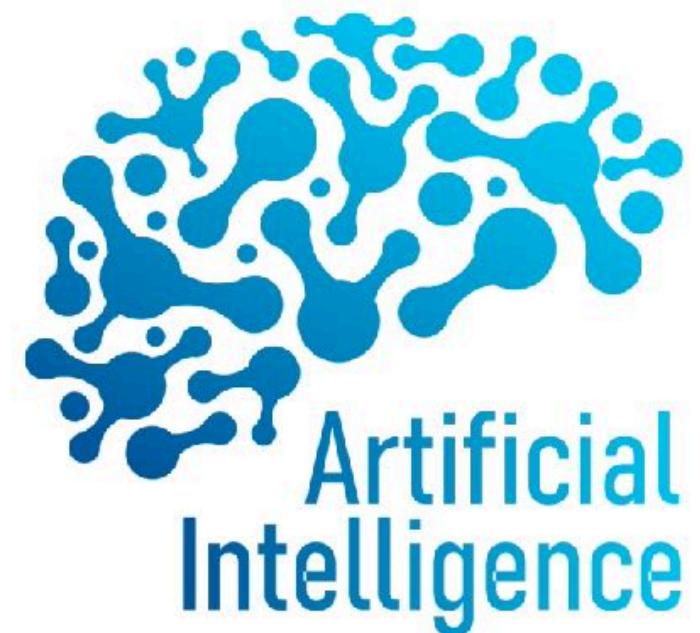


Was bedeutet das alles für Sie als Studierende?



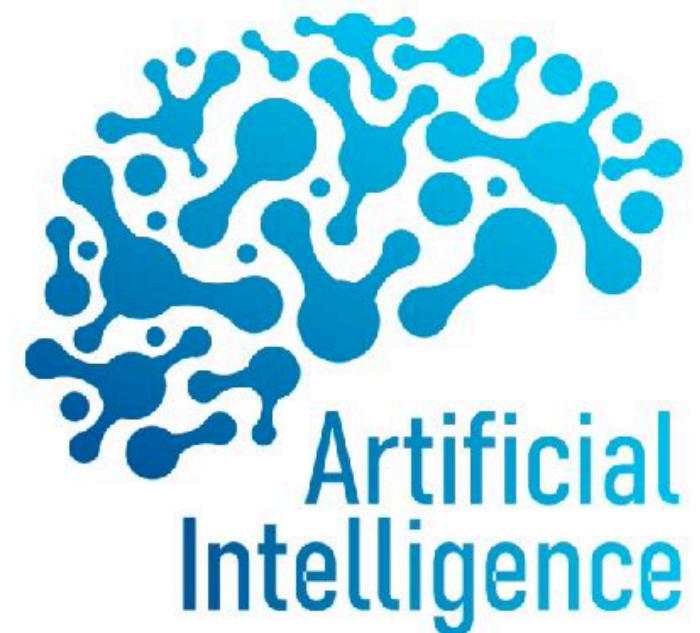
Vibe Coding

- ▶ Software-Entwicklung mit Hilfe von LLMs
- ▶ Simon Willison: “Vibe coding your way to a production codebase is clearly risky. Most of the work we do as software engineers involves evolving existing systems, where the quality and understandability of the underlying code is crucial.”



Vibe Coding - Probleme

- ▶ gut für Prototyping, enormer Aufwand bei Produktsystemen für Debugging, Testen und Kontrolle
- ▶ KI liefert eine 80%-Lösung, hoher Aufwand für die letzten 20%
 - ▶ **“There ain't no such thing as a free lunch”**
- ▶ Vibe Coding funktioniert, wenn man weiß, was man tut
- ▶ Was macht der generierte Code?
 - ▶ Compliance & Security
 - ▶ Was wenn es in 20 Jahren keine erfahrenen Entwickler mehr gibt, die in der Lage sind selbst Code zu schreiben?



Vibe Code **Fixers**

VibeCodeFixers

Find Experts

Join as Expert

Connect with Expert Developers to Fix Your Code

Find skilled developers who can fix bugs, optimize performance, and ensure your Vibe projects are secure and vulnerability-free

Browse Experts →

Become an Expert

Why Choose VibeCodeFixers?



Bug Fixing

Expert developers to identify and fix issues in your code



Security Analysis

Thorough vulnerability assessment and security improvements



Code Optimization

Enhance performance and implement best practices

A man with a beard and short brown hair, wearing a grey button-down jacket over a white t-shirt and blue jeans. He is looking directly at the camera with a stressed expression, his right hand pressed against his forehead. He is standing to the right of the website mockup, partially overlapping it.



Fragen wir mal jemanden, der sich damit auskennt

► ChatGPT, warum sollten Menschen weiterhin Programmieren lernen?

► **Grundverständnis der digitalen Welt**

► nur wer Programmieren versteht, kann digitale Werkzeuge hinterfragen und optimieren

► **Problemlösekompetenz**

► logisches Denken und strukturiertes Herangehen bei komplexen Aufgaben

► **Steuerung von KI & Technologien**

► KI-Systeme erfordern menschliche Kontrolle, Steuerung und Anpassung



Fragen wir mal jemanden, der sich damit auskennt

- ▶ ChatGPT, was passiert, wenn wir Menschen nicht mehr programmieren können?
- ▶ **Abhängigkeit:** Ihr seid abhängig von KI-Tools
- ▶ **Weniger Verständnis:** Ihr kennt lediglich die „Oberfläche“, nicht den Kern
- ▶ **Höheres Risiko:** Weniger Kontrolle über Qualität, Sicherheit und Datenschutz
- ▶ **Verpasste Chancen:** Fehlende Fähigkeit, eigene kreative Ideen technisch umzusetzen

Bottom Line KI-Assistenten



Man muss erst selbst programmieren lernen, bevor man KI-Assistenten **sicher**, **zielgerichtet** und **kompetent** einsetzen kann.

Materialien zu Vorlesung

- ▶ Vorlesungsfolien, Übungen, Literatur, Tutorials, Beispiele, Abgaben etc. in Moodle
 - ▶ <https://moodle.htwsaar.de/>
 - ▶ Kurs: PIB - Programmierung 2
 - ▶ **Bitte registrieren Sie sich für den Kurs!**
 - ▶ Benachrichtigungen zu Änderungen etc. werden via Moodle versendet
 - ▶ Registrierungscode: BaXu



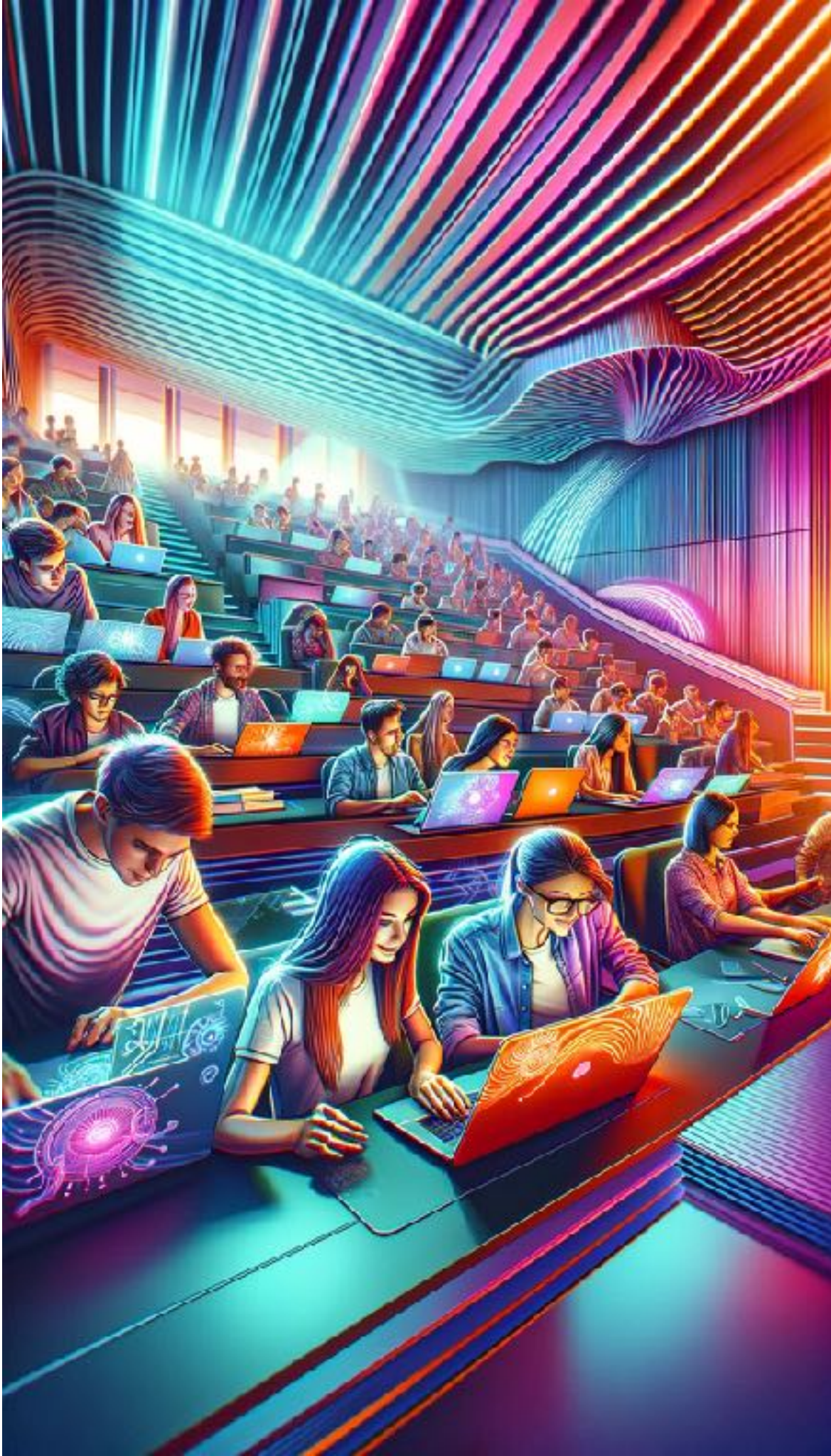
Machen Sie **Notizen**



Buzz **Group / Group** Work



10 Seconds



Coding **Sessions**

Bring
Your
Own
Device

Literatur



C. ULLENBOOM: Java ist auch eine Insel, Rheinwerk Computing, 18. Auflage, 2025

Umfassendes Nachschlagewerk

J. BLOCH: Effective Java: Third Edition, Addison-Wesley, 2018

Sammlung von Best Practices, Standardwerk



Literatur



R.-G. URMA, M. FUSCO, A. MYCROFT: Modern Java in Action: Lambdas, streams, functional and reactive programming, Manning, 2nd Edition, 2018

zu Kapitel 5 und 6

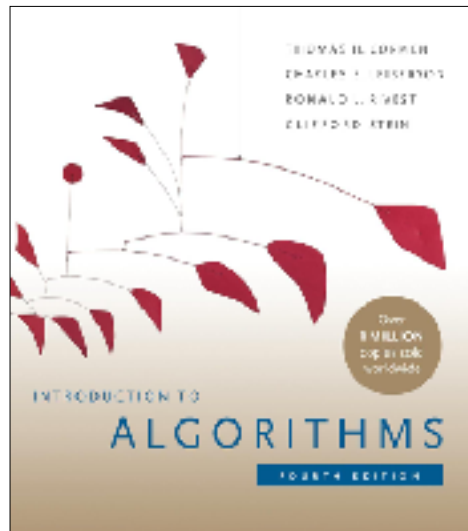
M. NAFTALIN, P. WADLER: Java Generics and Collections: Fundamentals and Recommended Practices, 2nd Edition, O'Reilly, 2025



zu Kapitel 4 und 6

Literatur

(Grundlagen zu Kapitel 6)



Th. CORMEN, Ch. LEISERSON, R. RIVEST:
Introduction to Algorithms, MIT Press, 4th Edition,
2022

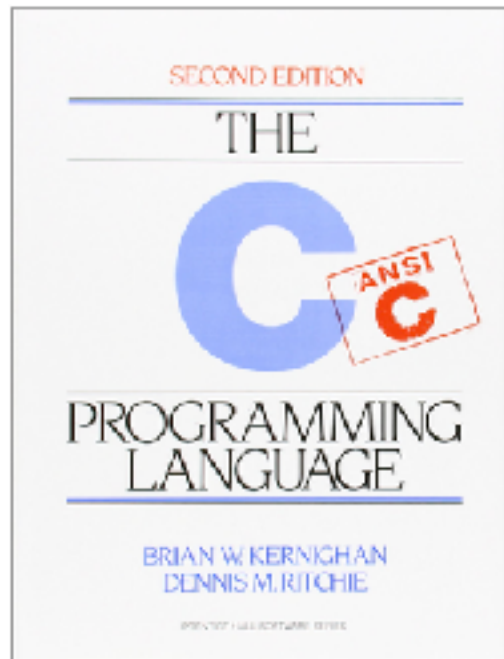
G. SAAKE, K.-U. SATTLER: Algorithmen und
Datenstrukturen: Eine Einführung mit Java,
dpunkt.verlag, 6. Auflage, 2020



A. SOLYMOSI, U. GRUDE: Grundkurs Algorithmen und
Datenstrukturen in JAVA: Eine Einführung in
die praktische Informatik, Springer, 2014

<http://dx.doi.org/10.1007/978-3-658-06196-8>

Literatur



B.W. KERNIGHAN, D. RITCHIE: The C Programming Language, Prentice Hall, 2nd Edition, 1988

Standardwerk

D. Logofatu: Einführung in C: Praktisches Lern- und Arbeitsbuch für Programmieranfänger, Springer, 2. Auflage, 2016

DOI: <http://dx.doi.org/10.1007/978-3-658-12922-4>





stöbern Sie in der Bib